Lab 07 Activity: NumPy & pandas on Soil Spectra

Marks: 20 (10 tasks × 2)
Submit: `<roll_no>.py` (e.g. `20231110XX.py`) on Moodle

1 Setup

You have been given the folder `setup/`:

```
setup/
|-- lab07.py            <- fill in the 10 functions here
`-- data/
    |-- spectra.csv     <- reflectance spectra, one row per sample
    `-- targets.csv     <- laboratory measurements
```

You need Python 3 with `numpy` and `pandas`. Work inside `setup/` and run `python lab07.py` to try your functions on the data.

> **This activity is autograded.** Keep the function names and parameters exactly as given in `lab07.py`, and *return* your answers (printing them is not enough). Submit only the single file, renamed to your roll number: `<roll_no>.py`, e.g. `20231110XX.py`.
>
> **The marks given by the autograder are final. No regrading requests will be entertained.** For example, if you rename a function or submit a file that does not run, and the autograder gives you 0, that 0 is your final mark.

2 The data

`data/spectra.csv`: 89 rows. The first column, `sample`, is the sample ID (`A1`, `B10`, `EC3`, …). The other 2 151 columns, named `350`, `351`, …, `2500`, are the reflectance (0–1) at that wavelength in nm.

`data/targets.csv`: the lab measurements. The sample ID is in the column `SOIL`. The columns you will use most are `CLAI` (clay, %), `SILT` (%), `SAND` (%) and `OC` (organic carbon, %). It also has the quirks of real-world data:

- missing values are written as a dash, –;
- the columns `CEC`, `CK` and `CCA` appear *twice* (two measurement methods); pandas renames the second copies `CEC.1`, `CK.1`, `CCA.1`;
- the two files are *not* in the same row order, and not every sample is in both files.

The dataset is synthetic, generated to behave like real soil spectra: soils rich in organic carbon are darker, water absorbs near 1414 and 1914 nm, and clay minerals absorb near 2206 nm.

In Tasks 7–10, `X` is a 2-D float array of shape (samples, wavelengths) and `wavelengths` is a 1-D int array with the wavelength (nm) of each column of `X`:

```
X = spectra.to_numpy()
wavelengths = spectra.columns.astype(int).to_numpy()
```

Your functions should work for *any* data in this format, not only the given files.

3 Part A: pandas

Task 1 `load_data(spectra_path, targets_path)` 2 marks

Read both CSV files and return the tuple `(spectra, targets)`.

- `spectra`: indexed by the sample ID (the `sample` column), with one float column per wavelength, named `'350'`, …, `'2500'`. Shape `(89, 2151)`.
- `targets`: keep the default integer index (0, 1, 2, …) and keep `SOIL` as a column. Every `'–'` must become `NaN`, so that all lab-property columns are numeric (float). Shape `(87, 18)`.

Hint: look at the `index_col` and `na_values` parameters of `pd.read_csv`.

Task 2 `missing_report(targets)` 2 marks

Return a `Series` indexed by the names of the columns that have *at least one* missing value, holding the number of missing values in each, sorted largest first.

```
>>> missing_report(targets).head(4)
CK.1     31
CCA.1    29
CEC.1    24
P        21
```

Columns with equal counts may appear in either order.

Task 3 `top_oc_samples(targets, n=5)` 2 marks

Return a `DataFrame` with *exactly* the columns `['SOIL', 'OC']` holding the `n` samples with the highest organic carbon, highest first. Ignore rows with a missing `OC`. The index does not matter.

```
>>> top_oc_samples(targets, 3)
   SOIL    OC
16   B8  7.02
57  EC5  6.71
9    B5  6.22
```

Task 4 `texture_class(targets)` 2 marks

Return a `Series` of strings, one per row of `targets` *with the same index*, applying these rules **in this order**:

`'unknown'`	if `CLAI` or `SAND` is missing
`'sandy'`	else if `SAND >= 70`
`'clayey'`	else if `CLAI >= 40`
`'loamy'`	otherwise

On the given data the counts are `loamy 40, clayey 27, sandy 12, unknown 8`. Note the boundaries: a sample with exactly `SAND == 70` is sandy. *Hint:* `np.select` or boolean masks with `.loc`.

Task 5 `site_summary(targets)` 2 marks

The *site* of a sample is the run of capital letters at the start of its `SOIL` ID: `'A1'` → `'A'`, `'EC12'` → `'EC'`. Return a `DataFrame` with one row per site, indexed by site name and sorted alphabetically, with exactly the columns:

`n`	number of rows (samples) of the site, *including* rows with missing values
`clay`	mean `CLAI`, ignoring missing values
`sand`	mean `SAND`, ignoring missing values
`oc`	mean `OC`, ignoring missing values

Round the means to 2 decimals.

```
>>> site_summary(targets).head(3)
       n   clay   sand    oc
site
A     12  29.85  41.66  2.37
B     12  56.92  18.19  2.87
C     10  19.95  61.95  0.92
```

Hint: `.str.extract(r"^([A-Z]+)", expand=False)`, then `groupby`/`agg`.

Task 6 `merge_spectra_targets(spectra, targets, cols)` 2 marks

Attach the lab properties named in the list `cols` (e.g. `['CLAI', 'OC']`) to each spectrum, keeping only the samples present in *both* tables. Return a `DataFrame`

- indexed by the sample ID (not 0, 1, 2, …),
- whose columns are all the spectra columns followed by `cols`, in that order, and no `SOIL` column.

With `cols=['CLAI', 'OC']` the result has shape `(86, 2153)`. Remember that the files are in different row orders.

4 Part B: NumPy

Everything in Part B can be written without a single Python loop. Try to.

Task 7 `strongest_absorption(X, wavelengths, lo, hi)` 2 marks

Consider only the wavelengths `w` with `lo <= w <= hi`. Return the wavelength (an `int` in nm, **not** an array position) at which the *mean spectrum* (the mean over all samples) is lowest. On the given data, `strongest_absorption(X, wavelengths, 1300, 2500)` is `1915`, the water band.
Careful: `np.argmin` on a sliced array returns a position *within the slice*. Do not assume the wavelengths start at 350 or are 1 nm apart.

Task 8 `standardize(X)` 2 marks

Return a *new* array of the same shape in which every column is standardised:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \qquad \mu_j = \text{mean of column } j, \quad \sigma_j = \text{std of column } j \text{ with } \texttt{ddof=0}.$$

Use the *population* standard deviation (`ddof=0`, NumPy's default; pandas' default is `ddof=1`). You may assume no column is constant. Do **not** modify `X`: `X -= ...` changes the caller's array. Each column of the result has mean 0 and standard deviation 1.

Task 9 `band_depth(X, wavelengths, left, centre, right)` 2 marks

The depth of an absorption band measures how far the reflectance at the band centre dips below a straight line (the *continuum*) drawn between two shoulder wavelengths. For each sample, with R_w its reflectance at wavelength w:

$$C = R_{\text{left}} + (R_{\text{right}} - R_{\text{left}})\,\frac{\text{centre} - \text{left}}{\text{right} - \text{left}}, \qquad \text{depth} = 1 - \frac{R_{\text{centre}}}{C}.$$

Return a 1-D array with one depth per sample (length `X.shape[0]`). `left < centre < right` are values present in `wavelengths`; find their columns by value, not by arithmetic on positions.

```
>>> band_depth(X, wavelengths, 2150, 2206, 2250)[:3]      # samples A1, A2, A3
array([0.0789, 0.0503, 0.114 ])                           # (rounded)
```

Sanity check: the clay band depth at 2206 nm correlates very strongly ($r \approx 0.98$) with `CLAI`.

Task 10 `target_correlation(X, y)` 2 marks

`y` is a 1-D float array with one lab value per row of `X`; it may contain `NaN`. Ignoring the samples where `y` is `NaN`, return a 1-D array of length `X.shape[1]` whose j-th entry is the Pearson correlation between column j of `X` and `y`:

$$r_j = \frac{\sum_i (x_{ij} - \bar{x}_j)(y_i - \bar{y})}{\sqrt{\sum_i (x_{ij} - \bar{x}_j)^2 \, \sum_i (y_i - \bar{y})^2}}.$$

Using `merged = merge_spectra_targets(spectra, targets, ['OC'])`, the inputs `merged[spectra.columns].to_numpy()` and `merged['OC'].to_numpy()` give $r \approx -0.919$ at 800 nm: organic carbon darkens the soil.
Hint: centre `X` and `y`, then use broadcasting (`y[:, None]`) and `.sum(axis=0)`.